

PROGRAMACIÓN

Los 5 Principios SOLID de P00

1. S - Single Responsibility Principle
2. O - Open/Closed Principle
3. L - Liskov Substitution Principle
4. I - Interface Segregation Principle
5. D - Dependency Inversion Principle

1.Principio de Responsabilidad Única:

Una clase debe tener una sola razón para cambiar, es decir, una sola responsabilidad.

2.Principio Abierto/Cerrado:

El software debe estar abierto para su extensión, pero cerrado para su modificación.

3.Principio de Sustitución de Liskov:

Una subclase debe poder sustituir a su clase base sin alterar el funcionamiento del programa.

4.Principio de Segregación de Interfaces:

Los clientes no deberían depender de interfaces que no usan.

5.Principio de Inversión de Dependencias:

Los módulos de alto nivel no deben depender de módulos de bajo nivel, ambos deben depender de abstracciones.

TDD

TDD (Test-Driven Development) es una metodología de desarrollo de software que se basa en escribir primero las pruebas antes del código que implementa la funcionalidad.

El ciclo de TDD es:

1. Escribir una prueba que falla (porque el código aún no existe o no está completo).
2. Escribir el código mínimo necesario para que la prueba pase.
3. Refactorizar el código si es necesario, manteniendo las pruebas verdes (que sigan pasando).

Este ciclo se llama Red-Green-Refactor:

- Red: la prueba falla.
- Green: el código pasa la prueba.
- Refactor: se mejora el código sin romper la funcionalidad.

Lenguajes de bajo nivel vs alto nivel

La diferencia entre lenguajes de bajo nivel y lenguajes de alto nivel está en el grado de abstracción que tienen con respecto al hardware (la máquina).

Lenguajes de bajo nivel:

- Están muy cerca del hardware.
- Tienen poca abstracción: el programador debe gestionar directamente memoria, registros, instrucciones de CPU, etc.
- Son más rápidos y eficientes, pero también más difíciles de aprender y usar.

Ejemplos:

- Lenguaje máquina (código binario que entiende la CPU directamente).
- Ensamblador (Assembly) (instrucciones simbólicas que representan operaciones del procesador).

Ventajas: Máximo control, alta eficiencia, ejecución muy rápida.

Desventajas: Muy complejos, poco portables entre diferentes máquinas.

Lenguajes de alto nivel:

- Están lejos del hardware y más cerca del lenguaje humano.
- Proveen alta abstracción, facilitando el desarrollo con estructuras fáciles de leer y escribir.
- Son portables entre diferentes sistemas.

Ejemplos:

- Python, Java, C#, JavaScript, PHP, Ruby, Go, etc.

Ventajas: Fáciles de aprender, más productividad, código legible y portable.

Desventajas: Menos control directo del hardware, menor velocidad en comparación con bajo nivel.

¿Por qué se llaman de bajo nivel si son más difíciles?

Se llaman de bajo nivel no por el nivel de dificultad, sino por el nivel de abstracción respecto al hardware:

- Bajo nivel (bajo grado de abstracción): poco abstracto, muy cerca de la máquina.

El programador tiene que pensar casi como piensa el procesador.

- Alto nivel (alto grado de abstracción): mucha abstracción, lejos del hardware.

El programador puede pensar en términos humanos (variables, objetos, funciones, etc.).

Ejemplo:

Para encender una lámpara:

- Lenguaje máquina (muy bajo nivel): 10110000 01100001 (puro binario).
- Assembly (bajo nivel): MOV AL, 61h una instrucción específica para el procesador.
- C (medio nivel): digitalWrite(pin, HIGH).
- Python (alto nivel): encender_luz().

Diferencias de saber un lenguaje de bajo nivel y pasar a alto nivel

Bajo nivel:

1. Entiendes cómo funciona realmente la máquina: memoria, registros, punteros, stack, CPU.
2. Desarrollas mentalidad de optimización y eficiencia.
3. Te acostumbras a pensar en pasos muy detallados.

Alto nivel:

Al tener esa base, los lenguajes de alto nivel resultan más fáciles, porque:

- Ya entiendes qué hay debajo de las abstracciones.
- Sabes por qué el lenguaje oculta detalles (manejo automático de memoria, garbage collector, etc.).
- Puedes escribir código más eficiente, aunque el lenguaje sea de alto nivel.

Ejemplo:

Si programas en C (bajo nivel), luego aprendes Python o JavaScript (alto nivel) se siente mucho más sencillo, porque esas cosas que el lenguaje hace por ti (manejo de memoria, string dinámicos, estructuras) ya sabes cómo funcionan internamente.